

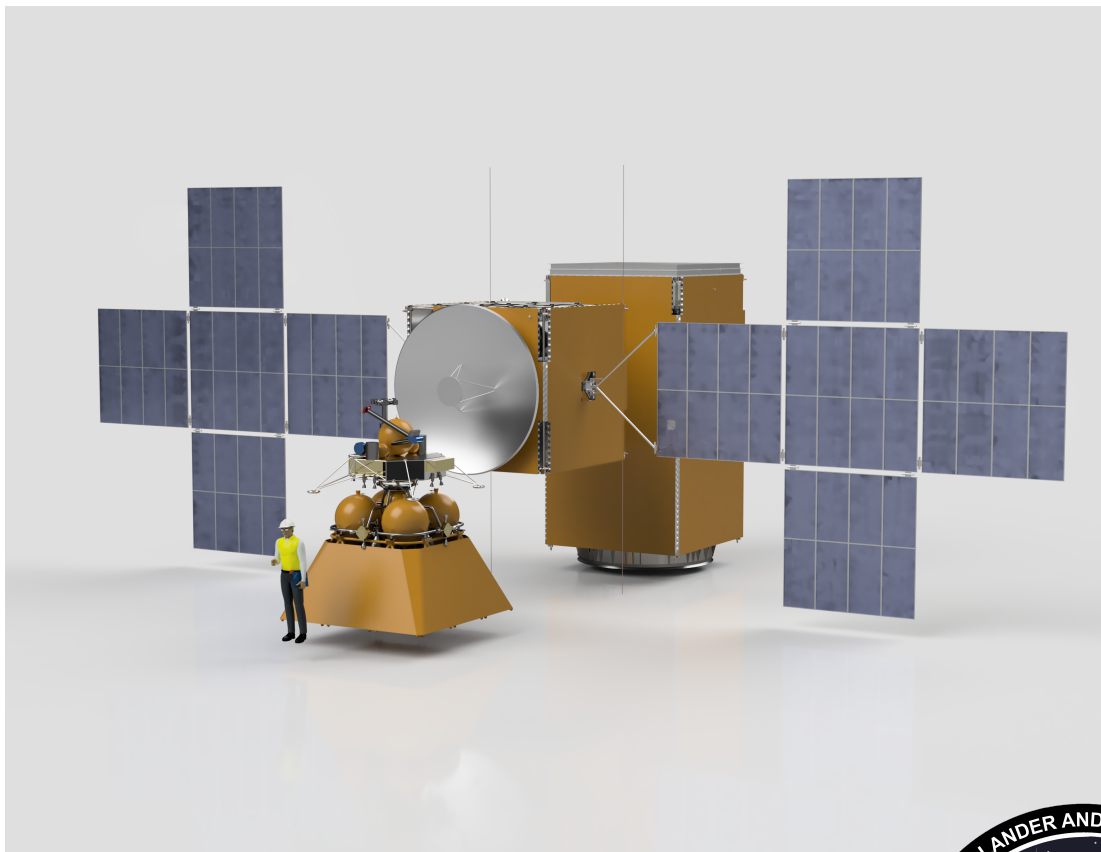
AERO60004 - AERONAUTICS GROUP DESIGN PROJECT

IMPERIAL COLLEGE LONDON

DEPARTMENT OF AERONAUTICS

ELMO - Europa Lander and Mapping Orbiter
Orbit Determination and Trajectory Optimisation
using Genetic Algorithms, Particle Swarm
Optimisation and the Nelder-Mead Algorithm

Supervisor: Prof. Luca Magri



Owain Evans

17 June 2024



Subteams and Members

Project Coordination	Jack Osborn Victor Djololian Charles Kempf
-----------------------------	--

Orbital Trajectory	Egor Kaygorodov Debdut Sengupta Owain Evans Yili Chen
---------------------------	--

Transfer Vehicle	Parth Gore Sriram Iyer Murray Doolan Jack Crane Léa Ruas
-------------------------	--

Lander	Klim Kapuka Miharu Aleks Kent Oliver Jackson Patrick Fan Jason Aretaois Hong Arthur Law
---------------	--

EDL	Jacob Dodridge Hengbo Cosmo Zhou Jirapat Rojnuckarin
------------	--

Communications	Andrei-Dinca Paduraru Lewis Mossaberi Ewan Sinclair Gaétan Bouscaillou
-----------------------	---

Abstract

The following report outlines the implementation of a generalised Lambert's problem solver and the optimisation schemes used to obtain an orbital trajectory for the ELMO mission. A hybrid optimisation scheme with a genetic algorithm performing global optimisation, and a Nelder-Mead algorithm performing local optimisation produced the best results. The final trajectory selected launches from Earth on 16/04/2031 on a Space-X Falcon Heavy and arrives at Europa orbit on 02/05/2036 utilising gravity assists from Mars, Earth, Ganymede and Europa. The final trajectory was selected to minimise the combined mass of propellant and radiation shielding; a ΔV budget of 2337 m/s with 13 m/s for end-of-life disposal was determined for the OTV.

Contents

List of Figures	iv
List of Tables	v
1 Introduction	1
2 Orbit Determination with Lambert's Problem	1
2.1 Implementation and Verification of a Generalised Lambert's Solver	1
2.2 Potential Improvements and Higher Fidelity Solver	2
3 Procedure for Trajectory Optimisation	2
3.1 Selection of optimisation schemes	2
3.2 Consideration and implementation of Hybrid algorithms	3
3.3 Optimisation approach, parameters and constraints	4
3.4 Hyperparameter tuning	4
3.4.1 GA (Genetic Algorithm)	5
3.4.2 PSO (Particle Swarm Optimisation)	6
3.4.3 Weighting on Arrival Hyperbolic Excess Velocity at Jupiter Encounter	7
4 Results	8
4.1 GA Results	8
4.2 PSO Results	9
4.3 Selected Trajectory	10
5 Verification of Final Trajectory with Porkchop Plots	10
6 Consideration of Finite Burn Losses	11
7 Summary and Areas for Improvement	12
A Lambert's Problem Geometry	15
B Calculation of Finite Burn Losses (FBL)	15
C Code for Lambert's Solver	16

List of Figures

1	Parameterised time of flight curves to verify the implementation of a generalised Lambert's solver	2
2	Reduction in ΔV and propellant mass due to hybrid optimisation	3
3	Impact of mutation probability on genetic algorithm (GA)	5
4	Impact of population size on genetic algorithm (GA)	5
5	Impact of populaion size and inertia weight on particle swarm optimisation (PSO)	6
6	Impact of learning rates on particle swarm optimisation (PSO)	7
7	Impact of weighting on the arrival velocity at Jupiter SOI, W_v on ΔV	7
8	Final ΔV and fitness for GA optimised trajectories	8
9	Porkchop (velocity contour) plots for final trajectory	11
10	Finite burn losses as function of T/W for thrust parallel to velocity [29]	12
11	Plot of final trajectory	12
12	Geometry of Lambert's classical problem [33]	15

List of Tables

1	Final hyperparameters for optimisation with GA/PSO	7
2	Table of considered assist trajectories	8
3	Table of median and upper/lower-quartile ΔV for combinations of trajectories	9
4	Table of median and upper/lower-quartile objective function for combinations of trajectories .	9
5	Table of minimum ΔV (m/s) and objective function fitness for combinations of trajectories from GA	9
6	Table of minimum ΔV (m/s) and objective function fitness for combinations of trajectories from PSO	10
7	Comparison of implemented optimisation schemes	10
8	A breakdown of the selected trajectory with associated burn ΔV s	10
9	Arrival and departure hyperbolic excess velocities for the E-M and G-U legs of the final trajectory	11

1 Introduction

For the proposed Europa Lander and Mapping Orbiter mission, it was necessary to determine a detailed trajectory to allow for sizing of the final spacecraft. The selected trajectory determined the total mission duration, fuel requirements and environmental extremes the spacecraft is exposed to. The work outlined in this report builds on the mission specification which summaries the conceptual design and preliminary trade studies for the mission [1]. It was decided the spacecraft would launch on Space-X Falcon Heavy into an Earth escape trajectory, utilising chemical propulsion to arrive in Europa orbit in less than 8 years from launch.

For past deep space missions to Jupiter and the outer Solar System, a series of gravity assists are commonly utilised to significantly reduce the required ΔV and propellant mass required, maximising the deliverable payload [2],[3]. The following report outlines the implementation of a Lambert's problem solver for orbital determination that was integrated with the work done by Sengupta.D [4], to form a low fidelity model for the purposes of analysis and optimisation of a gravity assisted trajectory. A number of optimisation approaches were considered as discussed in both the following report and Sengupta.D's report, and from the results the final trajectory was selected.

2 Orbit Determination with Lambert's Problem

To calculate the arrival and departure hyperbolic excess velocity for each leg of the trajectory, a Lambert's problem solver was implemented. Lambert's problem describes the determination of an orbit from the initial and final position vectors and the transfer time between the two points. The original formulation of the problem considers two possible trajectories, the "short-way" (corresponding to a transfer angle of $0 < \pi$) and the "long-way" (an angle of $\pi < 2\pi$), requiring a different formulation of the problem for each type of transfer orbit: elliptic, parabolic and hyperbolic. Such an implementation would severely limit the utility of the solver for application within the Jovian system, for which transfers of multiple orbital revolutions feature in the set of optimal transfer trajectories. Likewise, the solver must include the capability to deal with all possible types of orbit to enable the solution of the hyperbolic gravity-assist/flyby trajectories and to prevent issues when optimising with a poor initial guess.

2.1 Implementation and Verification of a Generalised Lambert's Solver

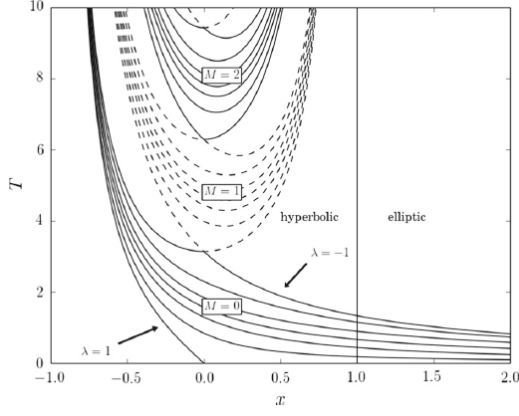
This necessitated the implementation of a generalised Lambert's solver as proposed by E.R.Lancaster et al [5] and R.H.Gooding [6], the solver implemented (code in Appendix C) follows an updated method outlined by D.Izzo [7]. The key difference for the generalised solver is the introduction of new non-dimensionalised parameters, x and λ , a non-dimesnionalised transfer time, T , and the number of complete revolutions, M . These parameters as defined in equations 1, 2 and 3, characterise all families of solutions; s , c and a are geometric properties of the orbit and are defined in convention with the classical Lambert's problem presented in appendix A.

$$\lambda^2 = \frac{s - c}{s} \quad (1)$$

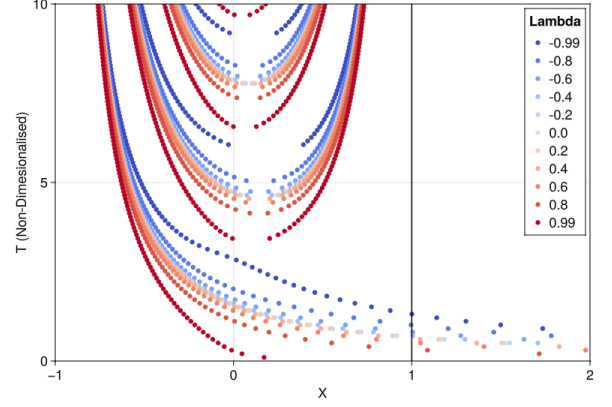
$$x^2 = 1 - \frac{s}{2a} \quad (2)$$

$$T = \sqrt{2 \frac{\mu}{s^3}} * (transfertime) \quad (3)$$

In order to solve the problem numerically Halley's/Householder methods were used as Newton's method does not converge for all cases, and the higher order solvers come at minimal computational expense. Derivatives up to the third order can call be computed via simple addition/subtraction and multiplication/division following one initial trigonometric evaluation as discussed in [7]. The final solver was verified for known trajectories by reproducing the orbital elements from the initial/final velocity vectors output. A plot of the parameterised time of flight curves for different values of x as presented in [7] was reproduced for direct comparison, shown in figure 1. The solver was finally integrated with the model developed by Sengupta.D [4].



(a) Time of flight curves parametrized using x for different λ and M values (Dario Izzo [7] Fig.2)



(b) Time of flight curves generated using implemented solver

Figure 1: Parameterised time of flight curves to verify the implementation of a generalised Lambert's solver

2.2 Potential Improvements and Higher Fidelity Solver

Due to the Keplerian, two-body assumptions made by Lambert's problem, the solver as implemented does not account for orbital perturbations such as solar radiation pressure or the influence of other bodies. This was deemed more than appropriate for initial trajectory determination, due to the limited information available from other sub-groups required to accurately calculate these perturbations. The potential to include perturbations using Sheperd's methods [8] was explored but the drastic increase in computational cost was considered extremely undesirable for initial optimisation.

3 Procedure for Trajectory Optimisation

3.1 Selection of optimisation schemes

For the initial trajectory optimisation a number of techniques were considered following a review of commonly used algorithms, a summary of popular approaches is outlined in [9]. It was found that for preliminary orbit determination a number of metaheuristics and evolutionary algorithms (EAs) are becoming increasingly common for impulsive trajectory design. As mentioned previously it was established that chemical propulsion was to be used [10], and so the optimisation techniques considered relate to high-thrust impulsive trajectories as opposed to low-thrust trajectories (common for electric propulsion). The algorithm's selected for implementation were a genetic algorithm (GA) and particle swarm optimisation (PSO), as discussed within this report, and differential evolution (DE) and artificial bee colony optimisation (ABC) as discussed in Sengupta.D's report [4]. All metaheuristics were implemented in Julia using the metaheuristics.jl package [11].

For the GA, real value encoding was used due to the nature of the inputs to the optimisation problem and the ease of integration. A polynomial mutation scheme was used in combination with simulated binomial crossover and for the selection method both generational-replacement and elitist-replacement were considered. It was determined that more optimal results were consistently produced when using an elitist-replacement strategy; whereby the best members of each population are retained from generation to generation, to ensure that the optimisation does not eliminate the current best until a new best solution is found. It is thought this was a result of the huge global scope of the optimisation problem, where the bounds used resulted in a large number of the population demonstrating a very poor fitness during the initial generations. Generational-replacement (where the entire population is replaced with offspring from the previous) struggled to consistently identify any one optimal trajectory and consequently struggled to converge in some cases.

The bounds used to define the scope of the optimisation were determined by considering past missions to Jupiter/the outer solar system for the gravity assisted-trajectories considered. Namely ESA Juice [12], NASA

JIMO [13], Europa Clipper [14] and the proposed Europa Orbiter/Lander missions [15], [16]. The selection of bounds was also aided using porkchop plots (discussed in more detail in section 5). Due to the limited data available for a very small number of past missions each with drastically different trajectories, it was decided that wide-bounds and a large search-space were required to ensure that for a given set of assists, suitable transfer opportunities could be found.

3.2 Consideration and implementation of Hybrid algorithms

To ensure the huge global scope of the search-space had a minimal impact on convergence, a number of hybrid optimisation schemes were considered including the possibility of combining the selected metaheuristic algorithms within a generalised island model. In this setup each optimisation scheme runs on its own "island" and members from each population are allowed to migrate to another "island" within an "archipelago" of directed graphs. This method is proposed by several sources [17] and [18] for trajectory optimisation with the potential to take advantage of parallel architectures. Another approach considered was to introduce a local-optimisation scheme in combination with the evolutionary algorithms [19], where local non-linear optimisation is performed for the best member of the final generation from the evolutionary algorithm. The use of gradient descent methods for local optimisation is suggested [20], however due to the nature of the model implemented, optimisation schemes requiring no derivatives were identified as better candidates, with Nelder-Mead being selected [21]. (Implementation was done with the NLOpt.jl package [22]). The improvement in results due to the hybrid optimisation scheme is presented in figure 2.

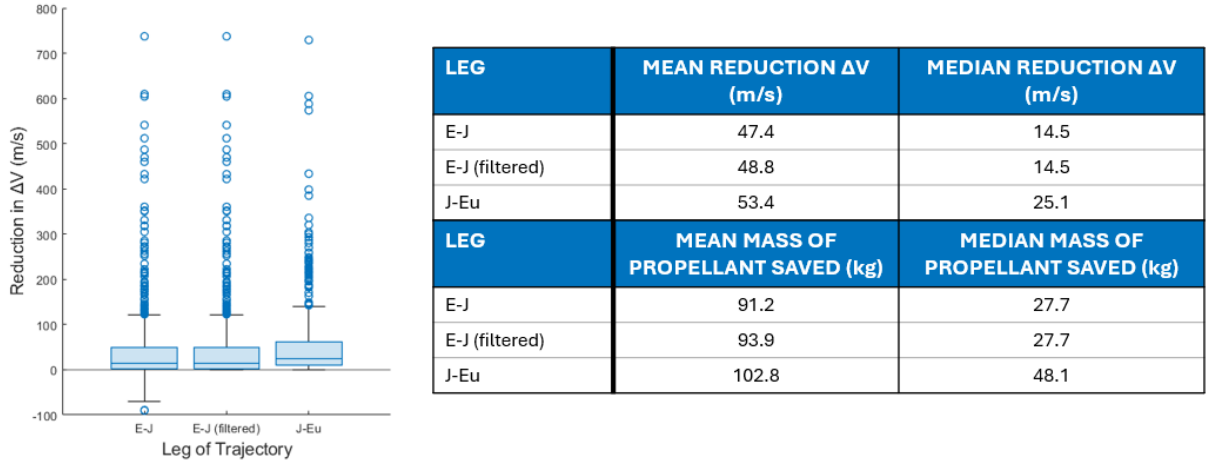


Figure 2: Reduction in ΔV and propellant mass due to hybrid optimisation

It can be clearly seen that for both legs a significant reduction in ΔV is observed with the mean value ranging between 48.8 and 53.4 m/s. This corresponded to significant propellant mass savings particularly for the Jupiter-to-Europa leg (J-Eu), with a mean/median saving of 102.8/48.1 kg. It was also found that in some cases for the Earth-to-Jupiter leg (E-J), the local optimisation resulted in an increase in the final ΔV . This was mitigated by comparing the final result before and after local optimisation and taking the best solution. It is thought this may be due to cases where the GA/PSO has already come extremely close to identifying the local minimum, and/or the region surrounding the result contains a large number of closely packed minima. Due to the ease of mitigating against these cases, this was not considered an issue. It was found that the main advantage of the hybrid scheme however, was its ability to aid in the convergence of the final result. By allowing the GA/PSO to act as a tool narrowing the bounds on the search-space for local optimisation, the computational expense of the optimisation process was significantly reduced. This is observed in the large number of outliers within figure 2 for both legs of the trajectory, indicative of such cases where the metaheuristic has struggled to converge within the time limit placed on the optimisation.

This was selected over the implementation of an island model which was deemed beyond the scope of the project. An island model would also be limited to just 2 or 3 different algorithms due to the computational

resources available, a 4 core system would require one core to initiate and perform the migration of populations, one core per island and potentially a single core to run background process unrelated to the optimisation, limiting the potential benefits.

3.3 Optimisation approach, parameters and constraints

For the optimisation, due to the computational cost of performing the optimisation for the full trajectory (from Earth to Europa) using the combined model discussed in Sengupta.D's report [4], it was decided that it would be resource efficient to attempt to optimise the trajectory as two separate problems. The final optimisation process was as follows:

1. Perform GA/PSO for the Earth-to-Jupiter leg
2. Perform Nelder-Mead optimisation locally for the best member of the population.
3. Input the arrival velocity and date at Jupiter SOI and repeat for the Jupiter-to-Europa leg

The main complication this approach introduces regards the arrival hyperbolic excess velocity of the spacecraft at Jupiter, which significantly impacts the ΔV (and fuel) required to slow the spacecraft and capture into a Jovian orbit. To account for this the objective function for the first optimisation problem was altered to include the weighted arrival velocity at Jupiter, W_v . The final objective function used to evaluate the trajectory is outlined by Sengupta.D in report [4] and is included below for the readers convenience as equation 4. The parameters R_x relate to the TID (total ionising dose) the spacecraft is exposed to and are defined fully in Sengupta.D's report. It should be noted that it was decided not to optimise for this objective function directly, as it was found to be too computationally intensive to compute the radiation dose for each member of the population each generation. Instead the Jupiter-to-Europa leg was optimised for ΔV before the radiation-dose and final objective function fitness was calculated. For the first leg from Earth-to-Jupiter the adapted objective function with the weighted arrival velocity is given in equation 5.

$$\left(m_{dry} \left[\exp \left(\frac{\Delta V}{I_{sp} g_0} \right) - 1 \right] \right) + (\rho_L A_L R_L + \rho_{ORB} A_{ORB} R_{ORB} + \rho_{OTV} A_{OTV} R_{OTV}) \quad (4)$$

$$\left(m_{dry} \left[\exp \left(\frac{\Delta V}{I_{sp} g_0} \right) - 1 \right] \right) + W_v (\|\vec{V}_{arrival}\| - 3000) \quad (5)$$

The parameters for which the optimisation was performed was limited to the transfer times of each orbit for a given set of gravity assists. It should be noted however that scope to expand the optimisation parameters to include variables such as the altitude of each flyby (which is related to the maximum delta v each assist can provide) was investigated as discussed in Sengupta.D's report [4].

A Constraint was placed on the optimisation for the total time from launch to arrival at Europa of 8 years (as defined in the mission statement/objectives [1]), as well as for the minimum distance to the sun due to thermal regulation requirements of the transfer vehicle sub-team [23]. Other groups were consulted with particular consideration given to the communications sub-team and the impact of the Earth-Jupiter distance on data-rate, however it was determined the distance can be easily accommodated with design changes of minimal impact to the wider spacecraft system/mission.

3.4 Hyperparameter tuning

Hyperparameter tuning was performed using a grid-search method. The grid-search was performed on a token trajectory, a Venus-Earth (E)VE(J), assist trajectory to Jupiter with a Ganymede assisted capture upon arrival (J)G(U). By separating the optimisation into two problems it was also possible to investigate the benefits of using a different combination of hyperparameters for each leg. The set of values for each hyperparameter evaluated during the grid search was taken to be a range centred around the values suggested by [18], with a population size of 200 and probability of mutation of 0.4 for the GA, and learning rates of 2.0 with an inertia weight of 0.8 the PSO [24].

3.4.1 GA (Genetic Algorithm)

For the GA the hyperparameters included the probability of mutation and population size. It can be seen in figures 3 and 4 that for several combinations of hyperparameters the most optimal solution did not differ significantly and so the selection of the final parameters for optimisation was primarily concerned with the median value and spread of results. This was motivated by the fact that the final trajectory selected was subject to the constraints discussed previously, which do not fully capture the complexity of all the design drivers at this stage in the design process. Therefore it was deemed advantageous to identify a large number of viable trajectories at this stage to allow for any future design changes.

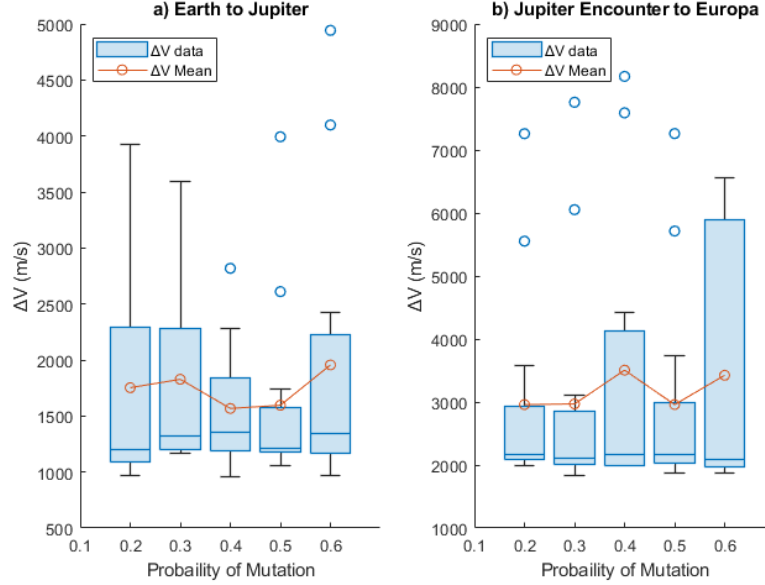


Figure 3: Impact of mutation probability on genetic algorithm (GA)

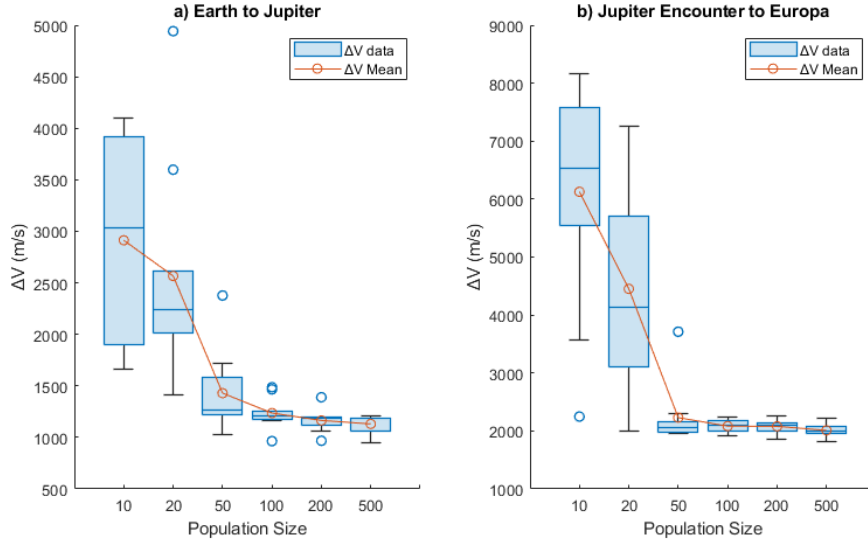


Figure 4: Impact of population size on genetic algorithm (GA)

For the Jupiter-to-Europa leg (J-Eu) the final value from the optimisation appeared less sensitive to population size than for the Earth-to-Jupiter leg (E-J) beyond a population size, $N=50$. This was somewhat unexpected since the number of transfer opportunities and the rate at which they recur is much greater within the Jovian system; due to the much shorter orbital periods of the Jovian moons compared to Venus, Mars and Jupiter. Combined with the suggestion that a lower probability of mutation, $p=0.3$ leads to more consistent results, it is thought that the number of assists is likely to influence which combination of hyperparameters performs

best. For the low number of assists in the token trajectory, it is possible that the limited number of dimensions of the optimisation problem, and hence reduced size of the search-space, leads to the results becoming less sensitive to population size and favours a lower probability of mutation. For both legs of the trajectory it was observed that increasing the population size beyond $N=200$ led to a very minor decrease in the median value and no improvement in the spread of results at vastly increased computational expense. For the probability of mutation, a value of $p=0.5$ saw a significant decrease in the spread of results for both legs compared to $p=0.4$ and $p=0.6$. Due to the fact that for the final optimisation a higher dimension problem is likely to be optimised for, $p=0.5$ was deemed best suited for the final optimisation.

3.4.2 PSO (Particle Swarm Optimisation)

For PSO, the hyperparameters included the inertial weight, the cognitive learning rate and the social learning rate. These rates inform how each member of the population moves through the search-space and can be altered to encourage faster/slower local or global convergence, with the benefit/expense of increasing/reducing the proportion of the search-space that is explored. The form of PSO implemented via the Metaheuristics.jl package [11] was limited to a constant inertial weight strategy, however due to the increased computational expense of adaptive schemes and the limited increase in accuracy as found by J.C.Bansal et al [25] this was considered acceptable. When compared to the GA, the PSO algorithm displayed a lesser sensitivity to population size in figure 5a, where for both legs of the trajectory N did not seem to have a significant impact on the most optimum value found. There was also little observable difference in the spread of results when going from $N=100$ to $N=200$ and so $N=100$ was selected to minimise the computational expense. It should be noted at this stage however that despite PSO achieving similar ΔV values for the Earth-to-Jupiter leg, it does not find any Jupiter-to-Europa legs of comparable ΔV , at best identifying trajectories of double the magnitude of the GA. For the inertia weight it was found that a value of $\omega=0.9$ produced the best results seen in figure 5b.

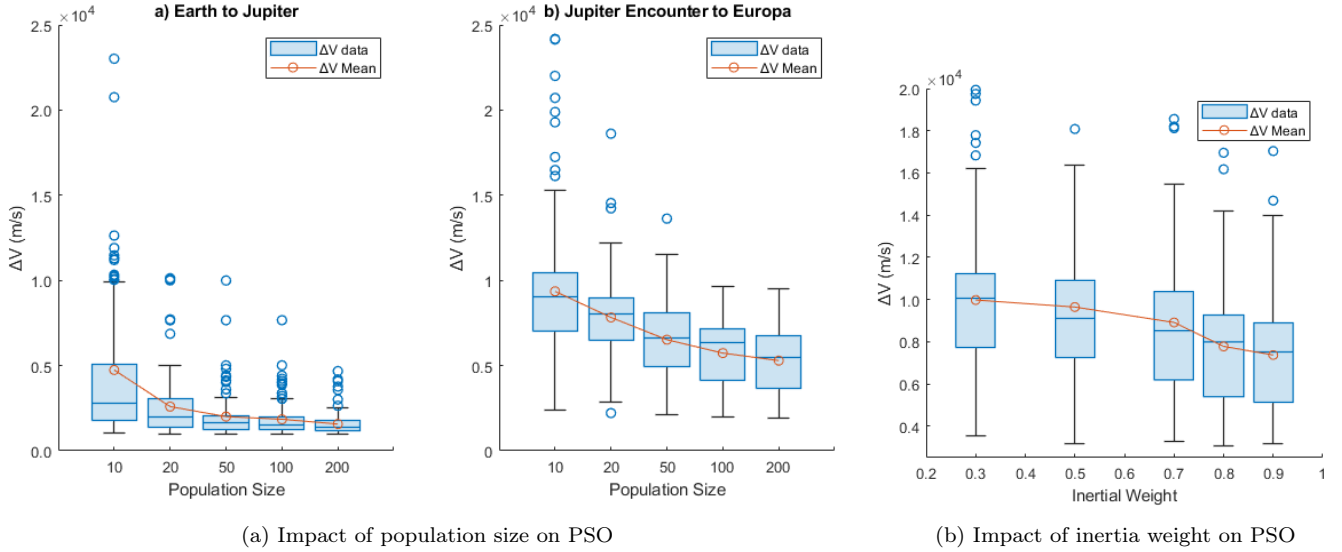


Figure 5: Impact of population size and inertia weight on particle swarm optimisation (PSO)

For the learning rates it was found that for the Earth-to-Jupiter leg a social learning rate C_2 much greater than the cognitive learning rate C_1 produced the best results 6. This may relate to the fact that for a given set of assists it is unlikely that more than a few optimal transfer windows exist for the Earth-to-Jupiter leg, and therefore it may be beneficial for members of the population to quickly converge towards the global optimum then perform a local optimisation on a smaller search-space. Conversely it was found that for the Jupiter-to-Europa leg a higher C_1 is preferable which would agree with the previous statement due to the recurring transfer windows within the Jovian system and likelihood several local optimum all correspond to potentially viable final trajectories. The final hyperparameters are given in table 1.

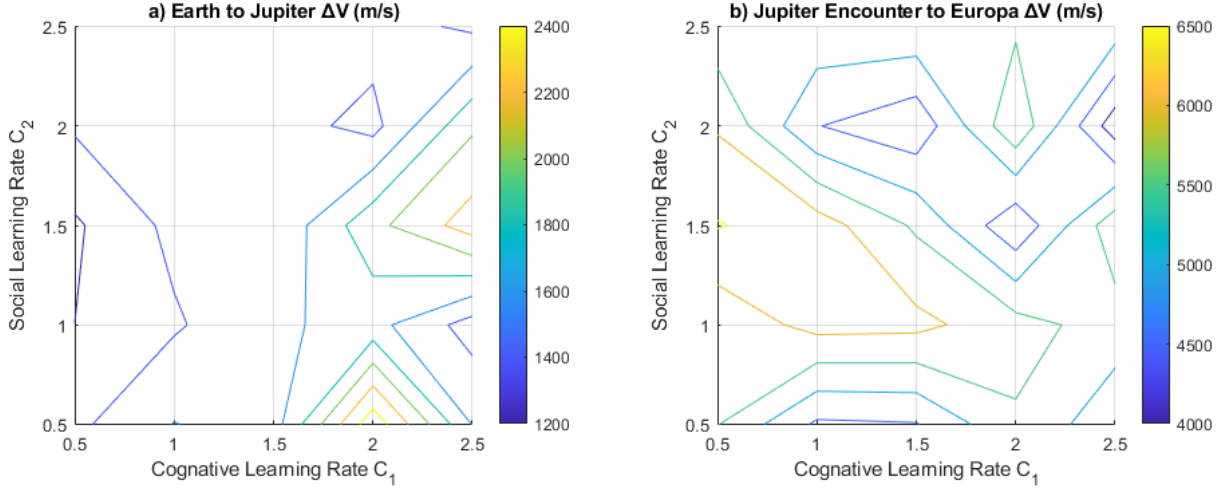


Figure 6: Impact of learning rates on particle swarm optimisation (PSO)

Table 1: Final hyperparameters for optimisation with GA/PSO

	Earth to Jupiter				Jupiter to Europa			
GA	N = 200	$P_{mutation} = 0.5$			N = 200	$P_{mutation} = 0.5$		
PSO	N = 100	$C_1 = 0.5$	$C_2 = 1.5$	$\omega = 0.9$	N = 100	$C_1 = 2.5$	$C_2 = 2.0$	$\omega = 0.9$

3.4.3 Weighting on Arrival Hyperbolic Excess Velocity at Jupiter Encounter

To tune the weighting (W_v in 5) on the arrival hyperbolic velocity at Jupiter, another grid-search was implemented with the selected hyperparameters for the GA. figure 7 shows that a noticeable decrease in the ΔV was observed for the Jupiter-to-Europa leg for weights of 0.5 and 0.6. Surprisingly however the median ΔV for the Earth-to-Jupiter leg only saw an increase for the 0.5 case and not for weights >0.5 . These two weights are demonstrated to be the best choices with a weight of 0.6 selected based on the quartiles for the total trip (combined) ΔV .

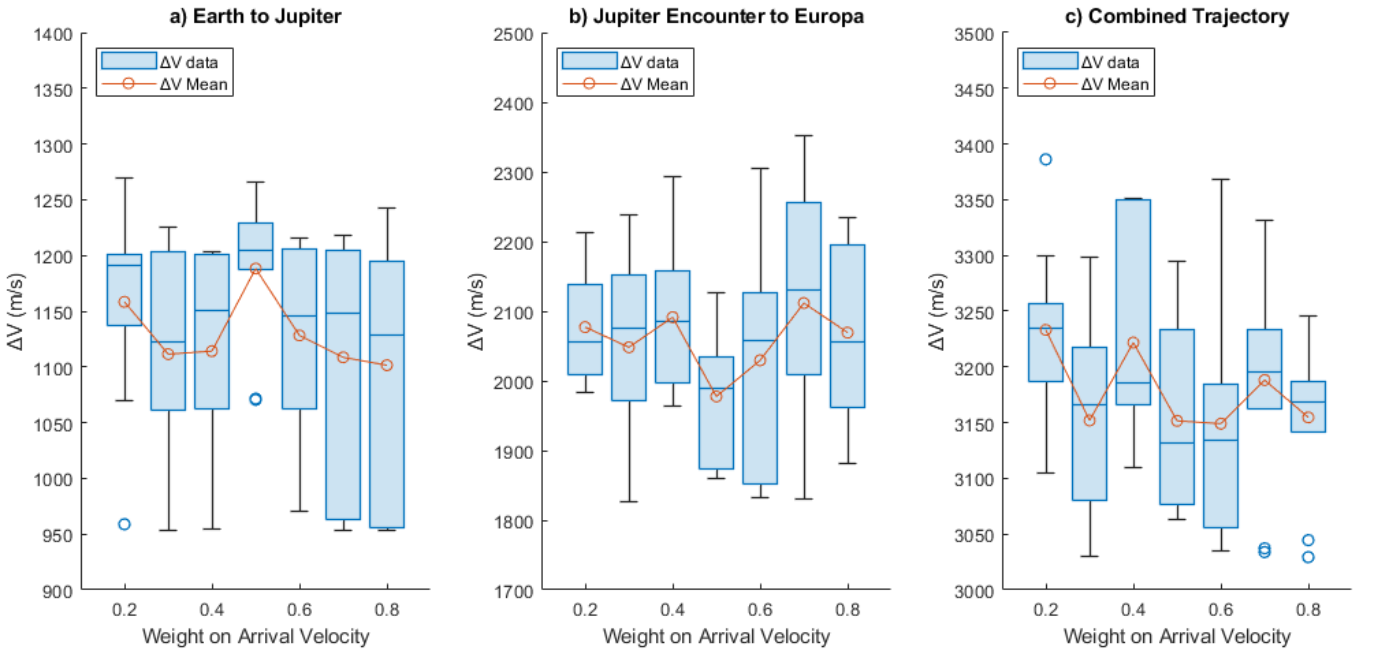


Figure 7: Impact of weighting on the arrival velocity at Jupiter SOI, W_v on ΔV

4 Results

For the selected hyperparameters the hybrid optimisation scheme was applied to a selection of initial trajectories as identified using the Tisserand plots and past mission trajectories as discussed in Kaygorodov.E's report [26]. The trajectories considered are given in table 2:

Table 2: Table of considered assist trajectories

Earth to Jupiter	Jupiter Encounter to Europa
(E)VE(J)	(J)CGUG(U)
(E)VEE(J)	(J)GCGCG(U)
(E)VEM(J)	(J)GUG(U)
(E)ME(J)	
V: Venus, E: Earth, M: Mars	C: Callisto, G: Ganymede, U: Europa

4.1 GA Results

The results from 600 optimised trajectories are presented in figure 8 for the aforementioned assists. From these results it was found that all combinations of assists produced feasible trajectories with minimum ΔV requirements of the same order of magnitude. The ΔV was found to be more heavily dependant on the assists within the Jovian system than the assists for the Earth-to-Jupiter leg. A noticeably higher spread in ΔV requirements for the (J)GCGCG(U) trajectory is likely due to the increased number of planetary bodies involved in the transfer and increased dimensions of the optimisation problem. In figure 8 the impact of the radiation exposure can be observed in the relative fitness of the (J)GCGCG(U) trajectory. Despite the higher median and minimum ΔV requirement as seen in figure 8, the absence of a Europa or Io assist prior to arrival at Europa leads to much higher orbits around Jupiter. Thus significantly reducing the absorbed radiation dose and leading to a competitive best/minimum fitness.

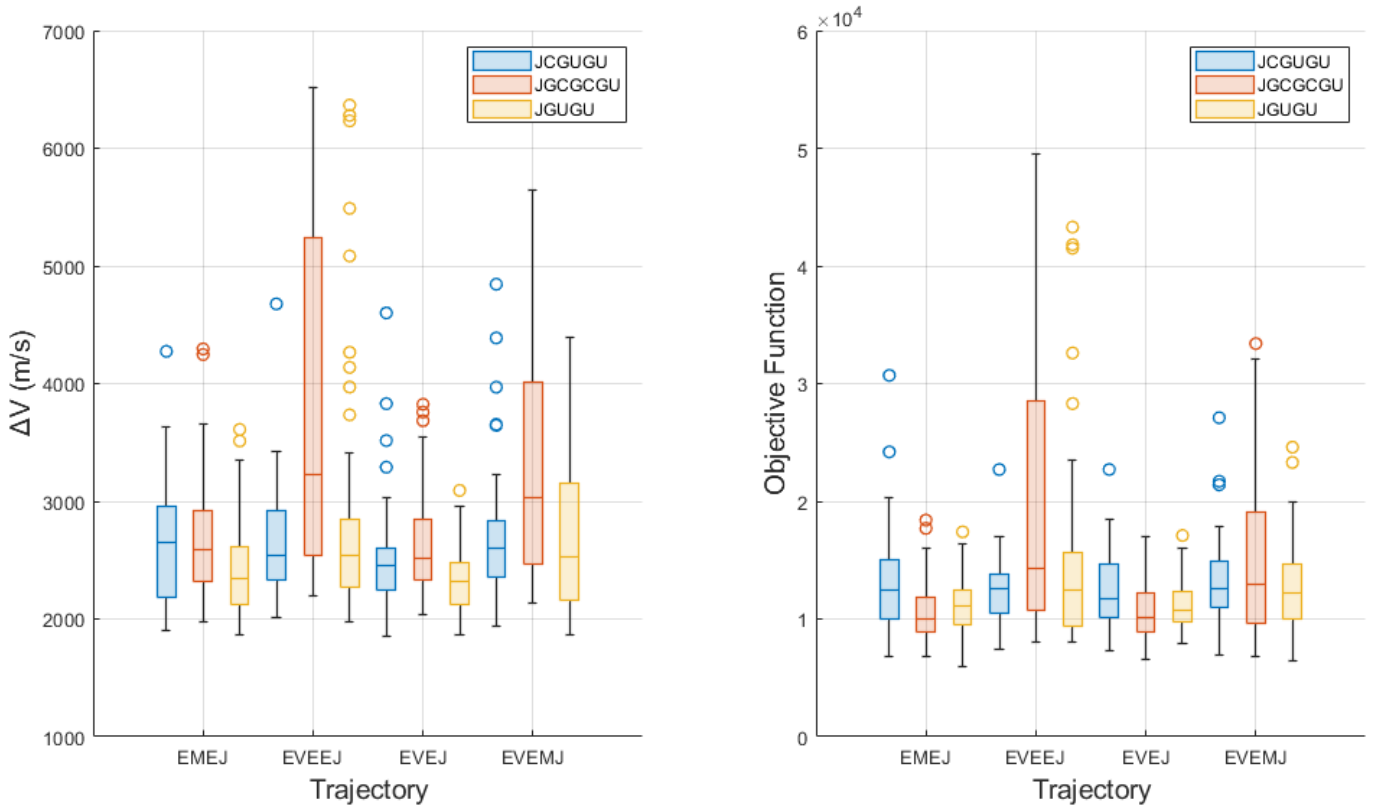


Figure 8: Final ΔV and fitness for GA optimised trajectories

The median and upper and lower quarterlies are summarised for the ΔV in table 3 and for the objective function fitness in table 4. For the both ΔV and fitness the median is consistently lower for the Earth-to-Jupiter trajectories (E)VE(J) and (E)ME(J). The trend in median values is likely related to the number of dimensions of the optimisation problem. For an increased number of planetary bodies, fewer favourable transfer windows are available leading to a greater spread in results. Considering the lower-quartiles it is found that the Jupiter-to-Europa leg (J)GUG(U) requires the least ΔV while (J)GCGCG(U) displays the best objective function fitness due to the lower radiation exposure. This is confirmed by the minimum values for the ΔV and objective function found for each trajectory given in table 5. As with the median values, the minimum ΔV is found to be consistently lower for the (E)VE(J) and (E)ME(J) trajectories, it is also found that the minimum objective function fitness is consistently higher for (E)VEE(J) compared to (E)VEM(J) which was not observed in the median values. (J)GCGCG(U) is again found to require the highest ΔV for the Jupiter-to-Europa leg with the reduced radiation exposure again resulting in a competitive fitness despite this. The lowest total ΔV is found for (E)VE(J)-(J)CGUG(U), 1856 m/s, and the best fitness is demonstrated by (E)ME(J)-(J)GUG(U), 5.92×10^3 .

Table 3: Table of median and upper/lower-quartile ΔV for combinations of trajectories

Earth-Jupiter Assists	Jupiter-Europa Assists		
	(J)CGUG(U)	(J)GCGCG(U)	(J)GUG(U)
(E)VE(J)	2456 (2246/2607)	2518 (2335/2842)	2314 (2129/2479)
(E)VEE(J)	2541 (2333/2919)	3225 (2545/5243)	2537 (2264/2843)
(E)VEM(J)	2597 (2350/2838)	3032 (2470/4015)	2533 (2158/3154)
(E)ME(J)	2647 (2188/2955)	2585 (2324/2918)	2347 (2123/2619)
ΔV (m/s) Median (Lower-Quartile/Upper-Quartile)			

Table 4: Table of median and upper/lower-quartile objective function for combinations of trajectories

Earth-Jupiter Assists	Jupiter-Europa Assists		
	(J)CGUG(U)	(J)GCGCG(U)	(J)GUG(U)
(E)VE(J)	1.17 (1.01/1.47)	1.01 (0.89/1.22)	1.08 (0.98/1.23)
(E)VEE(J)	1.26 (1.05/1.38)	1.43 (1.07/2.85)	1.25 (0.93/1.57)
(E)VEM(J)	1.26 (1.10/1.49)	1.29 (0.97/1.91)	1.23 (1.00/1.47)
(E)ME(J)	1.24 (1.00/1.50)	1.00 (0.89/1.18)	1.12 (0.95/1.24)
Objective Function ($\times 10^4$) Median (Lower-Quartile/Upper-Quartile)			

Table 5: Table of minimum ΔV (m/s) and objective function fitness for combinations of trajectories from GA

Earth-Jupiter Assists	Jupiter-Europa Assists		
	(J)CGUG(U)	(J)GCGCG(U)	(J)GUG(U)
(E)VE(J)	1856 0.730	2042 0.658	1864 0.792
(E)VEE(J)	2008 0.745	2191 0.809	1978 0.806
(E)VEM(J)	1940 0.688	2133 0.682	1868 0.649
(E)ME(J)	1908 0.683	1980 0.682	1863 0.592
Minimum ΔV (m/s) Objective Function ($\times 10^4$)			

4.2 PSO Results

For PSO the results are summarised in table 6. Due to the higher computational cost far fewer results were generated than with the GA (42 compared to 600). It was found that a number of results failed to converge within the time limit placed on the optimisation despite Nelder-Mead leading to a mean reduction in the final ΔV of 11.7% (in some cases a $>50\%$ reduction). Trajectories for which the solution did not converge to less than 10000 m/s ΔV are not included within the table, this highlights the huge cost of PSO where for the GA a time-constraint of 300 seconds was set for each leg of the trajectory with only 1.7% of results failing

to converge, compared to 40.5% of results for PSO. The best result found was worse than the median result produced by the GA for the same (E)VE(J)-(J)CGUG(U) trajectory, with a ΔV of 2517 m/s compared to 2456 m/s. In spite of this it can be seen that a fitness of 7.00×10^3 was achieved, a very comparable result to the best fitness found by the GA. This suggests that while PSO was found to be too computationally expensive given the resource available for this project, it still has the potential to produce valuable results for optimisation problems of this a nature.

Table 6: Table of minimum ΔV (m/s) and objective function fitness for combinations of trajectories from PSO

Earth-Jupiter Assists	Jupiter-Europa Assists		
	(J)CGUG(U)	(J)GCGCG(U)	(J)GUG(U)
(E)VE(J)	2517 0.700	2728 0.955	4028 1.161
(E)VEE(J)	9132 4.035	3387 1.009	–
(E)VEM(J)	–	–	5868 1.910
(E)ME(J)	–	3965 1.282	8513 5.861
Minimum ΔV (m/s) Objective Function ($\times 10^4$)			

4.3 Selected Trajectory

The results from the optimisation processes were compared to the results from the optimisation performed by Sengupta.D [4] in table 7. It was found that GA and DE performed best with the computational intensity of PSO and ABC proving to be disadvantageous; the comparison is based on the minimum ΔV found rather than the objective function as the optimisation was not performed due to high computational cost. Once taking radiation exposure into account, the best trajectory was found to be an (E)ME(J)-(J)GUG(U) trajectory produced by the GA/Nelder-Mead hybrid scheme with an objective function of 5.92×10^3 and ΔV of 1997 m/s. A breakdown of the final trajectory is presented in table 8 with a launch date of 06/04/2031 and arrival in Europa orbit on 02/05/2036, well within the time constraint of 8 years. The use of a Mars-Earth gravity assist (MEGA) was also of great benefit to the OTV design, due to the significantly reduced thermal load when compared to a Venus assisted trajectory [23]. (with a minimum distance to the Sun of 1 au, well in excess of the 0.5 au constraint). The use of Ganymede to capture at Jupiter is observed to be similar to other proposed optimal trajectories as presented in [27].

Table 7: Comparison of implemented optimisation schemes

Optimisation Scheme	GA	PSO	DE	ABC	LBFGS
Best ΔV (m/s)	1856	2517	1906	5268	251900

Table 8: A breakdown of the selected trajectory with associated burn ΔV s

Encounter	Date	Burn	ΔV (m/s)	Encounter	Date	Burn	ΔV (m/s)
Earth (launch)	2031 APR 16	YES	1048.5	Jupiter SOI	2035 DEC 09	YES	128.4
Mars	2032 APR 07	NO	0.0	Ganymede	2036 FEB 29	NO	0.0
Earth	2033 MAY 17	NO	0.0	Europa	2036 APR 17	NO	0.0
				Ganymede	2036 APR 30	NO	0.0
				Europa (arrival)	2036 MAY 02	YES	820.0
Total			1048.5	Total			948.4

5 Verification of Final Trajectory with Porkchop Plots

To perform a quick verification of the results from the optimisation by ensuring the Lambert’s problem solver and the low fidelity model were integrated correctly, Porkchop plots were used. These plots show the departure and arrival hyperbolic excess velocities for a transfer between two planetary bodies given a departure and

arrival date. As such they were also useful in determining bounds for transfer times within the optimisation problems. For each leg the arrival and departure velocities as output by the low fidelity model are compared to the point on the porkchop plots corresponding to the arrival and departure dates. The final Earth-Mars leg is presented in figure 9a along with the final Ganymede-Europa leg in figure 9b and the corresponding arrival and departure velocities for both legs in table 9. It was observed that in both cases the plots agree with the output from the model.

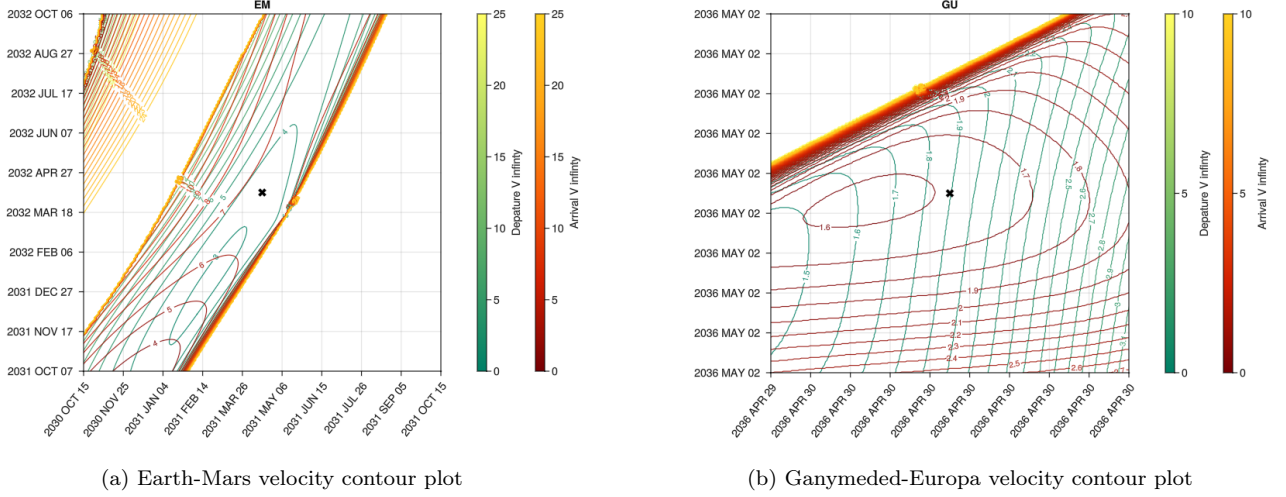


Figure 9: Porkchop (velocity contour) plots for final trajectory

Table 9: Arrival and departure hyperbolic excess velocities for the E-M and G-U legs of the final trajectory

	Earth-Mars (E-M)	Ganymede-Europa (G-U)
Departure Velocity (km/s)	3.300	1.895
Arrival Velocity (km/s)	6.667	1.610

6 Consideration of Finite Burn Losses

As previously mentioned and discussed in Sengupta.D's report [4]. The assumption of impulsive burns will result in an underestimate for ΔV , the additional ΔV required can be broken into two components. The ΔV wasted to accelerate the spacecraft radially (not in the desired direction) due to the orientation of the thrust vector throughout the burn, and Oberth "losses". The Oberth effect describes the increased efficiency of a burn when performed at the perapsis of a given orbit as an object falls deeper into a gravity well [28], for a finite length burn it is impossible to apply the total impulse at the most efficient point in space and so a further "loss" is incurred. The size of this loss/the additional ΔV required is heavily dependant on the burn duration/thrust-to-weight (T/W) of the spacecraft as well as the attitude control scheme used throughout each burn. Consequently high fidelity models are required to determine an accurate value for finite burn losses (FBL) with the estimated value determined based on the numerical/analytical results presented within [29]. The T/W of the s/c was provided by the OTV design team [30], and a constant pro-grade burn was assumed. This resulted in an estimated value of 200 m/s for the 1048.5 m/s burn in table 8 taken from figure 10, given a T/W of 0.029 in Earth's SOI. For the final 820.0 m/s burn at Europa the T/W is 0.214 and so finite burn losses are comparably negligible. Based on an analytical estimate [31], it can also be shown that if this initial burn was to be moved outside of Earth's SOI the finite burn loss could be reduced to 1.536×10^{-5} m/s (equations for this approximation are provided in appendix B). Additional sources of ΔV were also considered, notably the requirement for trajectory correction manoeuvres (TCMs) as discussed by Sengupta.D [4] totaling 140 m/s and a 13 m/s deorbit burn required by the OTV at Europa. This was calculated assuming a single impulse burn once in the final science orbit to lower the perapsis to the 155 km altitude identified by Chen.Y [32], allowing orbital perturbations to crash the OTV into the surface.

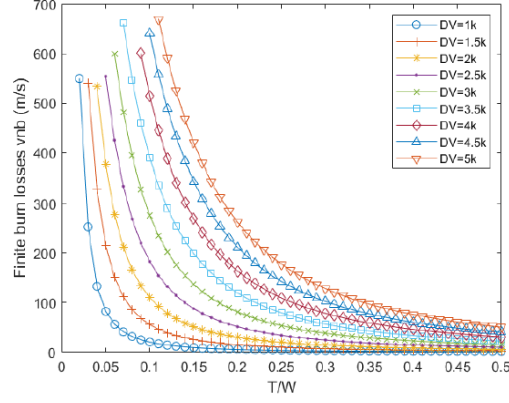


Figure 10: Finite burn losses as function of T/W for thrust parallel to velocity [29]

7 Summary and Areas for Improvement

Following the optimisation process outlined above a final trajectory was identified with the aim of minimising the total spacecraft mass, considering propellant and the mass of radiation shielding. This was found to be an (E)ME(J) (MEGA) trajectory from Earth-to-Jupiter, with a (J)GUG(U) series of assists in the Jovian system. This resulted in a total ΔV of 2337(+13) m/s for the OTV, (1997 m/s, plus 200 m/s FBL and 140 m/s TCM prior to separation from the orbiter/lander, and 13 m/s post separation to deorbit). During the transfer, a minimum distance to the sun of 1 au is maintained with the total transfer lasting 5 years 16 days. The selected trajectory satisfied all requirements and budgets as set by the other sub-teams and project coordinators.

It is noted that no capture burn is performed at Jupiter, rather a burn is applied upon arrival at Jupiter's SOI before a gravity assisted capture at Ganymede. The burn at Jupiter SOI is required to couple the two optimisation problems and is noted as a key area from improvement, where the location of this burn if performed as a TCM closer to/further away from Ganymede is likely to reduce the total ΔV . In this vein the next steps would be to input the selected trajectory into a higher fidelity model, accurately modelling the TCM and FBL as mentioned above, accounting for orbital perturbations. Other model specific improvements are discussed by Sengupta.D [4]. Regarding the optimisation approach itself, the biggest constraining factor was the available computational resource, with improved capabilities, greater flexibility would be afforded to allow for refinement of the identified trajectory within a higher fidelity model. During this project the scope was developed to optimise the entire trajectory as one problem incorporating the radiation model developed by Kaygorodov.E [26]. Due to the limited computational resource, the final approach necessitated significant compromise. A plot of the final optimised trajectory is provided below in figure 11.

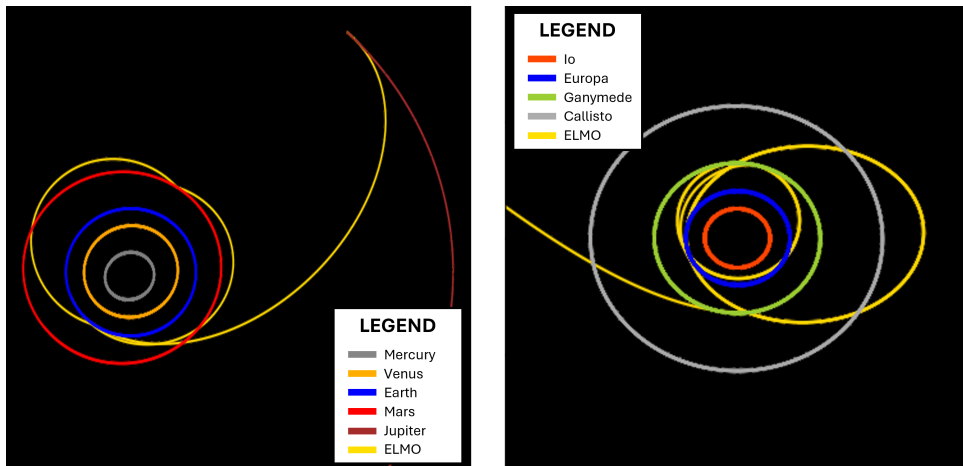


Figure 11: Plot of final trajectory

References

- [1] Victor Djololian. *Project Coordination, Astrobiological Potential and Planetary Protection*. Imperial Collge London - Department of Aeronautics, June 2024.
- [2] P. Tan-Wang J. Krajewski J. Dooley. *Europa Lander ICEE-2 Proposal Information Package*. July 2018.
- [3] Masataka Okutsu et al. *Low-Thrust Trajectories to Jupiter via Gravity Assists from Venus, Earth, and Mars*. Jan. 2010.
- [4] Debdut Sengupta. *On the Optimisation and Analysis of the Earth-Europa Transfer Orbit for the Europa Lander and Mapping Orbiter*. Imperial Collge London - Department of Aeronautics, June 2024.
- [5] E.R.Lancaster and R.C.Blanchard. “A Unified Form of Lambert’s Problem”. In: *NASA TN D-5368* (Sept. 1969).
- [6] R.H.Gooding. *A Procedure for the Solution of Lambert’s Orbital Boundary-Value Problem*. Royal Aerospace Establishment, Sept. 1990.
- [7] Dario Izzo. “Revisiting Lambert’s problem”. In: *Celestial Mechanics and Dynamical Astronomy* 121.1 (Oct. 2014), p. 1. DOI: 10.1007/s10569-014-9587-y.
- [8] Stanley W. Shepperd. “UNIVERSAL KEPLERIAN STATE TRANSITION MATRIX”. In: *Celestial Mechanics* 35 (1985).
- [9] Runqi Chai et al. “A review of optimization techniques in spacecraft flight trajectory design”. In: *Progress in Aerospace Sciences* 109 (June 2019). DOI: 10.1016/j.paerosci.2019.05.003.
- [10] Charlie Kempf. *An investigation into the Risks and Failure modes of a Europa Orbilander Mission*. Imperial Collge London - Department of Aeronautics, June 2024.
- [11] Jesús-Adolfo Mejía-De-Dios and Efrén Mezura-Montes. *Metaheuristics: A Julia Package for Single- and Multi-Objective Optimization*. Oct. 2022. DOI: 10.21105/joss.04723.
- [12] “JUICE Science Management Plan”. In: *ESA/SPC 1* (June 2012).
- [13] Dave Edwards. *An Overview of the Jupiter Icy Moons Orbiter (JIMO) Mission, Environments, and Materials Challenges*. NASA Flight Mechanics and Analysis Division, Sept. 2012.
- [14] Brent Buffington. “TRAJECTORY DESIGN FOR THE EUROPA CLIPPER MISSION CONCEPT”. In: *American Institute of Aeronautics and Astronautics* (2014).
- [15] K. Clark et al. *Return to Europa: Overview of the Jupiter Europa orbiter mission*. -04-24 2010. DOI: 10.1016/j.asr.2010.04.011.
- [16] Robert Dean Abelson and James H. Shirley. “Europa Lander Mission”. In: *Space Technologies and Applications International Forum*. NASA JPL, Feb. 2005.
- [17] Matteo Rosa Sentinella and Lorenzo Casalino. “Hybrid Evolutionary Algorithm for the Optimization of Interplanetary Trajectories”. In: *Journal of Spacecraft and Rockets* 46.2 (Mar. 2009), p. 365. DOI: 10.2514/1.38440.
- [18] Eric Woods. “ORBITAL OPTIMIZATION OF INTERPLANETARY TRAJECTORIES WITH ENVIRONMENTAL PERTURBATIONS”. MA thesis. June 2018.
- [19] Ya-Zhong Luo, Guo-Jin Tang, and Hai-Yang Li. “Optimization of multiple-impulse minimum-time rendezvous with impulse constraints using a hybrid genetic algorithm”. In: *Aerospace Science and Technology* 10.6 (Feb. 2006), p. 534. DOI: 10.1016/j.ast.2005.12.007.
- [20] Nathan L. Parrish and Daniel J. Scheeres. “LOW-THRUST TRAJECTORY OPTIMIZATION WITH SIMPLIFIED SQP ALGORITHM”. In: *AAS* 17-832 (2015).
- [21] S. Singer and J. Nelder. “Nelder-Mead algorithm”. In: *Scholarpedia* 4.7 (2009). revision #91557, p. 2928. DOI: 10.4249/scholarpedia.2928.
- [22] Steven G. Johnson. *The NLOpt nonlinear-optimization package*. <https://github.com/stevengj/nlopt>. 2007.
- [23] Léa Ruas. *Thermal Control and Power Design of ELMO’s Transfer Vehicle and Orbiter*. Imperial Collge London - Department of Aeronautics, June 2024.
- [24] James Kennedy and Russell Eberhart Washington. *Particle Swarm Optimization*. 1995.
- [25] J. C. Bansal et al. “Inertia Weight Strategies in Particle Swarm Optimization”. In: *IEEE* (2011).
- [26] Egor Kaygorodov. *The Development of a Jovian Radiation Model For Direct Use in Orbital Trajectory Optimisation*. Imperial Collge London - Department of Aeronautics, June 2024.
- [27] Damon Landau, Nathan Strange, and Try Lam. “SOLAR ELECTRIC PROPULSION WITH SATELLITE FLYBY FOR JOVIAN CAPTURE”. In: *AAS* 10-169 (2010).

- [28] Philip Rodriguez Blanco and Carl E. Mungan. “Rocket Propulsion, Classical Relativity, and the Oberth Effect”. In: *The Physics Teacher* 57.7 (Oct. 2019), p. 439. DOI: 10.1119/1.5126818.
- [29] José Carlos et al. *Finite burn losses in spacecraft maneuvers revisited*. Tecnico Lisboa, Nov. 2020.
- [30] Murray Doolan. *Propulsion System for the ELMO Transfer Vehicle and Orbiter*. Imperial Collge London - Department of Aeronautics, June 2024.
- [31] Howard M. Robbins. “An analytical study of the impulsive approximation.” In: *AIAA Journal* 4 (1966), pp. 1417–1423. URL: <https://api.semanticscholar.org/CorpusID:122598086>.
- [32] Chen Yili. *The Design and Analysis of ELMO’s Science Orbit Around Europa*. Imperial Collge London - Department of Aeronautics, June 2024.
- [33] D.Amato. *Orbital Mechanics Lecture Notes: Lecture 6*. Imperial College London, 2024.

A Lambert's Problem Geometry

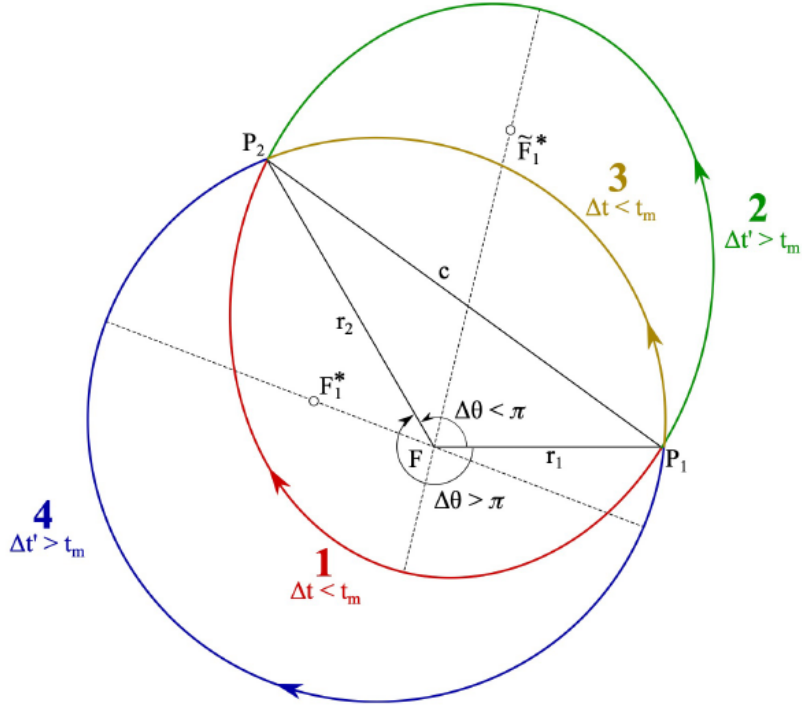


Figure 12: Geometry of Lambert's classical problem [33]

$$s = \frac{r_1 + r_2 + c}{2} \quad (6)$$

B Calculation of Finite Burn Losses (FBL)

$$FBL \leq \frac{1}{24} (\omega_s * (burntime))^2 \Delta V \quad (7)$$

$$\omega_s^2 = \frac{\mu}{r^3} \quad (8)$$

$$burntime = \frac{I_{sp}(1 - \exp(\frac{-\Delta V}{I_{sp}g}))}{T/W} \quad (9)$$

C Code for Lambert's Solver

```
# Function to solve lambert's problem, method used is outlined by Lancaster and Blanchard:
# https://ntrs.nasa.gov/api/citations/19690027552/downloads/19690027552.pdf
# the paper presents a unified form of the problem which is valid for elliptic, hyperbolic
# and parabolic trajectories

using LinearAlgebra
using SPICE
using HypergeometricFunctions

function SolveLamberts(R1, R2, TransferTime, GM)

    Arcs = ["SW", "LW"]

    # Initialise empty arrays
    V1 = []
    V2 = []
    a = []
    e = []
    TLP = []
    p = []

    for j in 1:2
        Direction = Arcs[j]

        # Determine geometric properties of triangle of R1, R2 and the central body
        r1 = norm(R1)
        r2 = norm(R2)
        c = norm((R1-R2))
        s = (1/2)*(r1+r2+c)

        # Find radial unit vectors at R1 and R2 (used to later determine velocities)
        i1r = R1/r1
        i2r = R2/r2
        ih = cross(i1r, i2r)
        ih = ih/norm(ih)

        # Find lambda (in place of finding central angle)
        lambdaSq = 1-c/s
        lambda = sqrt(lambdaSq)

        # Determine direction of travel for short-way/long-way solution (captured in sign of lambda)
        if Direction == "LW"
            lambda = -lambda
            i1t = cross(i1r, ih)
            i2t = cross(i2r, ih)
        else
            i1t = cross(ih, i1r)
            i2t = cross(ih, i2r)
        end

        # Compute non-dimensional Time, Tnd
        Tnd = sqrt(2*(GM/s^3))*TransferTime
```

```

function findT(xnew,lambda,M)
    ynew = sqrt(1-lambda^2*(1-xnew^2))
    Flag = 0
    if xnew > 1
        coshphi = xnew*ynew-lambda*(xnew^2-1)
        if coshphi < 1
            println("Warning: domain error for cosh with: ",coshphi)
            coshphi = 1
        else
            end
        phi = acosh(coshphi)
    elseif -1 <= xnew <= 1
        cosphi = xnew*ynew+lambda*(1-xnew^2)
        if cosphi < -1
            println("Warning: domain error for cos with: ",cosphi)
            cosphi = -1
        else
            end
        phi = acos(cosphi)
    else
        Tnew = -1; TnewPrime = 0; Tnew2Prime = 0; Tnew3Prime = 0;
        Flag = 1
    end
    if 0.999 < xnew < 1.001 && Flag == 0
        Tnew = (2/3)*(1-lambda^3)
        TnewPrime = (2/5)*(lambda^5-1)
        Tnew2Prime = 0
        Tnew3Prime = 0
    elseif sqrt(0.6) < xnew < sqrt(1.4) && Flag == 0
        # Bounds as empiracally suggested from paper
        eta = ynew-lambda*xnew
        S1 = (1/2)*(1-lambda-xnew*eta)
        Q = (4/3)*pFq((3,1),(5/2, ),S1)
        Tnew = (eta^3*Q+4*lambda*eta)/2
        TnewPrime = (3*Tnew*xnew-2+2*lambda^3*xnew/ynew)/(1-xnew^2)
        Tnew2Prime = (3*Tnew+5*xnew*TnewPrime+2*(1-lambda^2)*lambda^3/ynew^3)/(1-xnew^2)
        Tnew3Prime = (7*xnew*Tnew2Prime+8*TnewPrime-6*(1-lambda^2)*lambda^5*xnew/ynew^5)/...
            ... (1-xnew^2)
    elseif Flag == 0
        Tnew = (1/(1-xnew^2))*((phi+M*pi)/(sqrt(abs(1-xnew^2)))-xnew+lambda*ynew)
        TnewPrime = (3*Tnew*xnew-2+2*lambda^3*xnew/ynew)/(1-xnew^2)
        Tnew2Prime = (3*Tnew+5*xnew*TnewPrime+2*(1-lambda^2)*lambda^3/ynew^3)/(1-xnew^2)
        Tnew3Prime = (7*xnew*Tnew2Prime+8*TnewPrime-6*(1-lambda^2)*lambda^5*xnew/ynew^5)/...
            ... (1-xnew^2)
    end
    return (Tnew, TnewPrime, Tnew2Prime, Tnew3Prime)
end

function Householder(x0,lambda,M,T)
    tol = 1
    iterations = 0
    xnew = x0

```

```

while tol > 0.00001      # tolerance from new paper
    (Tnew, TnewPrime, Tnew2Prime, Tnew3Prime) = findT(xnew,lambda,M)
    if Tnew == -1
        return xnew = -100
    else
    end
    iterations = iterations + 1
    f = (Tnew-T)
    f1 = TnewPrime
    f2 = Tnew2Prime
    f3 = Tnew3Prime
    xnew2 = xnew-f*((f1^2-f*f2/2)/(f1*(f1^2-f*f2)+f3*f^2/6))
    tol = abs(f)
    xnew = xnew2
    if iterations >= 1000
        #println("Reached maximum number of iterations, Transfer time did not converge")
        #println("x = ", xnew)
        break
    end
end
return xnew
end

function Halley(x0,lambda,M)
    tol = 1
    iterations = 0
    xnew = x0
    while tol > 0.00001
        (Tnew, TnewPrime, Tnew2Prime, Tnew3Prime) = findT(xnew,lambda,M)
        if Tnew == -1
            return xnew = -100
        else
        end
        iterations = iterations + 1
        f = TnewPrime
        f1 = Tnew2Prime
        f2 = Tnew3Prime
        xnew2 = xnew-(2*f*f1)/(2*f1^2-f1*f2)
        tol = abs(f)
        xnew = xnew2
        if iterations >= 1000
            # println("Reached maximum number of iterations, Inital guess for x not found")
            # println("x0 = ", xnew)
            break
        end
    end
    return xnew
end

function Findxy(lambda, T)
    Mmax = floor(T/pi)
    if Mmax > 5
        Mmax = 5
    end
end

```

```

end
TSingleRev = acos(lambda)+lambda*sqrt(1-lambda^2)
T0 = TSingleRev+Mmax*pi

# Find minimum time to achieve transfer to determine
number of solutions to Lambert's problem
if T < T0 && Mmax > 0
    # Find Tmin
    xin = 0
    xmin = Halley(xin,lambda,Mmax)
    if xmin == -100
        x = -100; y = -100
        return (x,y)
    else
    end
    ymin = sqrt(1-lambda^2*(1-xmin^2))
    if xmin > 1
        phi = acosh(xmin*ymin-lambda*(xmin^2-1))
    else
        phi = acos(xmin*ymin+lambda*(1-xmin^2))
    end
    Tmin = (1/(1-xmin^2))*((phi+Mmax*pi)/sqrt(abs(1-xmin^2))-xmin+lambda*ymin)

    # Adjust Mmax from Tmin found
    if Tmin > T
        Mmax = Mmax-1
    end
end

# Find T1 when x = 1
T1 = (2/3)*(1-lambda^3)

# Find initial guess for x (Based on value of T1)
if T >= T0
    xin = (T0/T)^(2/3)-1
elseif T < T1
    xin = (5/2)*(T1*(T1-T))/(T*(1-lambda^5))+1
elseif T1 < T < T0
    xin = (T0/T)^(log2(T1/T0))-1
end

# Find x using iterative solver
x1 = Householder(xin,lambda,0,T)
if x1 == -100
    y1 = -100
else
    y1 = sqrt(1-lambda^2*(1-x1^2))
end
append!(x,[x1])
append!(y,[y1])

while Mmax > 0
    # Find initial guesses for x

```

```

xin1 = (((Mmax*pi+pi)/(8*T))^(2/3)-1)/(((Mmax*pi+pi)/(8*T))^(2/3)+1)
xin2 = (((8*T)/(Mmax*pi))^(2/3)-1)/(((8*T)/(Mmax*pi))^(2/3)+1)

# Find x using iterative solver for both initial guesses
x1 = Householder(xin1,lambda,Mmax,T)
if x1 == -100
    y1 = -100
else
    y1 = sqrt(1-lambda^2*(1-x1^2))
end
x2 = Householder(xin2,lambda,Mmax,T)
if x2 == -100
    y2 = -100
else
    y2 = sqrt(1-lambda^2*(1-x2^2))
end

append!(x,[x1, x2])
append!(y,[y1, y2])
Mmax = Mmax-1
end

return (x, y)
end

# Call functions to determine transfer trajectories
x = []
y = []
(x, y) = Findxy(lambda, Tnd)

# Find velocities and orbital elements
# -----
# Define intermediate values
A = sqrt((GM*s)/2)
B = (r1-r2)/c

# Method 2 to avoid rounding error
thetar = 2*acos((lambda*s)/(sqrt(r1*r2)))
C = 2*((r1*r2)/c^2)^(1/2)*sin(thetar/2)

# Loop through all possible transfers
for i in eachindex(x)
    # Calculate radial and tangential velocity vectors
    if x[i] == -100
        vr1 = 100000
        vr2 = 100000
        vt1 = 100000
        vt2 = 100000
    else
        vr1 = A*((lambda*y[i]-x[i])-B*(lambda*y[i]+x[i]))/r1
        vr2 = -A*((lambda*y[i]-x[i])+B*(lambda*y[i]+x[i]))/r2
        vt1 = A*C*(y[i]+lambda*x[i])/r1

```

```

        vt2 = A*C*(y[i]+lambda*x[i])/r2
    end

    vnew = vr1*i1r+vt1*i1t
    append!(V1,[vr1*i1r+vt1*i1t])
    append!(V2,[vr2*i2r+vt2*i2t])

    H1 = cross(R1,vnew)
    hmag = norm(H1)
    append!(a,[s/(2*(abs(x[i]^2-1)))])
    append!(p,[(hmag^2)/GM])
    E1 = (1/GM).*(cross(V1[end],H1)-GM.*R1/r1)
    append!(e,[norm(E1)])
    if E1[2] > 0
        append!(TLP,[acos(E1[1]/e[end])])
    else
        append!(TLP,[2*pi-acos(E1[1]/e[end])])
    end

end

end

return [V1, V2, a, e, TLP, p]
end

```